Backpropagation and Computational Graphs

Scaler School of Technology | Deep Learning I | Week 2 — Session 2/3 | 8 May 2026

What Is This Lecture About?

Session 2 answered *how* a neural network computes predictions (the forward pass). Session 3 answers *how* the network **learns** — by flowing gradients backward through the network to update every weight. We'll build up from the chain rule to computational graphs to the full backpropagation algorithm, and see how PyTorch automates it all.

1 The Chain Rule & Why We Need It

Single-variable and multi-variable chain rule. Local gradients — each step is simple. How the chain rule connects the loss to every weight through layers of composition. Worked example: nested function derivatives.

2 Computational Graphs

Representing any computation as a directed acyclic graph (DAG). Nodes = operations, edges = data flow. Forward pass evaluates left to right. Backward pass propagates gradients right to left. Worked example: $f(x, y) = (x + y) \cdot (x - y)$.

3 Backpropagation & Gradient Flow

The full algorithm: forward pass → compute loss → backward pass → update weights. Worked numerical example on a 2-2-1 MLP (continuing from Session 2). Vanishing/exploding gradients. Why ReLU fixes vanishing gradients. PyTorch autograd.

What to Expect in the Session

- Chain rule refresher with worked examples
- Drawing and evaluating computational graphs
- Full backward pass on the Session 2 MLP
- Every weight gets a gradient — by hand
- Vanishing gradients: sigmoid vs. ReLU
- PyTorch autograd: `loss.backward()`
- 2 short quizzes + a final 5-question quiz
- Case study: one training step for MNIST

How to Prepare (Optional but Helpful)

- Review Session 2's forward pass (especially the 2-2-1 MLP example)
- Recall the chain rule from calculus: $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$
- Remember what a derivative measures (rate of change)
- Optional: watch 3Blue1Brown "Backpropagation calculus" (YouTube)
- Bring a pen and paper for live gradient computation
- Ensure Google Colab access with GPU runtime

Duration: ~2 hours **Prerequisites:** Session 2 (forward pass), basic calculus (chain rule) **Instructor:** Priyansh Saxena